HM-02 | Phase 2 Extension

Phase 2 Guide

Check a patient's whole history across several doctors

> **What Phase 2 adds:** (1) merge prescriptions from three doctors into one timeline and flag overlaps and cross-doctor duplicates; (2) re-check doses when a new kidney value arrives; (3) mark unknown brands as unverified or propose a likely composition with confidence; (4) rank alerts so the most urgent appear first.
>
> **Your Phase 1 files are untouched.** Phase 2 is three new files that build on `engine.py`, so your earlier walkthroughs and line numbers stay correct.

1. Requirement map

Phase 2 bullet	What we built	Where
Merge prescriptions from three doctors into one timeline; flag overlaps and cross-doctor duplicates	Each medicine becomes a 'course' with doctor, start date and end date. The history is cut into periods where the set of active medicines is constant; each period is checked. A duplicate is reported only while courses overlap in time, and names the doctors	phase2.py: make_course, check_timeline, _segment_alerts; app2.py three doctor boxes and timeline grid
Re-check doses when a new kidney-function value arrives	Kidney readings (date + creatinine) feed a Cockcroft-Gault estimate. Each period uses the latest reading on or before it. recheck_with_new_value runs the history with and without the newest reading and lists the alerts the new value caused	phase2.py: creatinine_clearance, RENAL_RULES, recheck_with_new_value; app2.py 'Kidney re-check'
Mark unknown brands as unverified, or propose a likely composition with confidence	resolve_medicine gives each medicine a status: verified, generic, proposed (with confidence) or unverified. Unverified medicines get a MAJOR alert and the overall banner becomes INCOMPLETE. Also fixes the Dolo 500 bug	phase2.py: resolve_medicine; app2.py banner and medicines table
Rank alerts by severity so the most urgent appear first	Alerts are sorted by severity, then by the earliest date at that severity, numbered #1, #2, ... and the same alert across many periods is merged into one line with its dates	phase2.py: check_timeline (ranked list); app2.py 'Alerts, most urgent first'

2. New files and how to run them

File	Lines	Job
phase2.py	239	Phase 2 brain: durations, kidney maths, resolving medicines, timeline check, re-check
app2.py	116	Phase 2 screen: three doctor boxes, kidney readings, timeline, ranked alerts, re-check
evaluate_phase2.py	86	22 automatic checks for the Phase 2 features

Put the three files in the same folder as engine.py, then:

```
cd Desktop\HM-03
```

```
py -3.13 evaluate_phase2.py          (should end with: 22/22 checks passed)
py -3.13 -m streamlit run app2.py    (opens the Phase 2 app)
```

> **Honest note on testing:** I ran the logic and the 22 checks in my environment and executed app2.py against a stand-in for Streamlit, but I could not open it in a real browser. Please run it on your laptop in the first 15 minutes, click through every box, and tell me if anything errors. All medicine rules, including the kidney thresholds, are illustrative placeholders that your pharmacist must confirm.

3. The key idea: courses and periods

In Phase 1 a prescription was one flat list. In Phase 2 each medicine is a **course** with a doctor, a start date and an end date (or no end = ongoing). The system cuts the patient's history at every date where something changes (a course starts, a course ends, or a new kidney value arrives). Inside each resulting **period** nothing changes, so we can run the same Phase 1 `analyse()` on exactly the medicines that are active.

```
Dr A  Combiflam  TDS   [1 Oct ====== 5 Oct]
Dr A  Glycomet   BD    [1 Oct ============================> ongoing
Dr B  Voveran    BD             [3 Oct ====== 7 Oct]
Dr B  Dolo 650   TDS             [3 Oct ====== 7 Oct]
Dr C  Ecosprin   OD                  [4 Oct ======================> ongoing
Kidney value               [1 Oct: creat 1.0]        [6 Oct: creat 2.1]

Change dates:  1  3  4  6  8 (Combiflam ended 5 -> next day 6 ...)

Period 1-2 Oct : Combiflam + Glycomet              -> paracetamol 975 mg
Period 3 Oct   : + Voveran + Dolo 650              -> paracetamol 2925 mg: cross-doctor duplicate
Period 4-5 Oct : + Ecosprin, Telma                 -> aspirin + ibuprofen interaction
Period 6-7 Oct : Combiflam ended; new kidney value  -> CrCl 28: metformin and diclofenac CRITICAL
```

Why this beats merging everything into one list

- A duplicate only matters if both medicines are taken **at the same time**. Dolo 650 from Dr A in week 1 and Combiflam from Dr B in week 3 is **not** a duplicate, and Phase 2 does not flag it (this is tested).
- Daily totals are correct per period, so a dose limit is only exceeded on the days it is really exceeded.
- A new kidney value changes what is safe **from its date onward**, which is exactly what 're-check' means.

What 'cross-doctor' adds

When a duplicate or interaction involves medicines from different doctors, the alert says which doctors, and for interactions adds *'different doctors, so neither may have seen both'*. That is the real-world danger Phase 2 is about.

4. phase2.py, section by section

4.1 Kidney rules and the kidney maths (lines 9-44)

```
 9  # ---------- 1. KIDNEY RULES (placeholders for pharmacist review) ----------
10  # ingredient -> rules in ASCENDING order of creatinine clearance (mL/min): (below, severity, message)
11  _NSAID = [(30, "CRITICAL", "Kidney clearance is below 30 mL/min: NSAIDs are usually avoided."),
12            (60, "MODERATE", "Reduced kidney function: use NSAIDs with caution and monitor.")]
13  RENAL_RULES = {
14      "metformin": [(30, "CRITICAL", "Metformin is not recommended below 30 mL/min kidney clearance."),
15                    (45, "MAJOR", "Kidney clearance below 45 mL/min: metformin dose needs review.")],
16      "ibuprofen": _NSAID, "diclofenac": _NSAID, "aceclofenac": _NSAID,
17      "amoxicillin": [(30, "MAJOR", "Kidney clearance below 30 mL/min: amoxicillin dose needs adjusting.")],
18      "clavulanic acid": [(30, "MAJOR", "Kidney clearance below 30 mL/min: amoxicillin-clavulanate dose needs  ...
19      "hydrochlorothiazide": [(30, "MODERATE", "May work poorly when kidney clearance is below 30 mL/min.")],
20  }
21
22  def creatinine_clearance(age, weight_kg, scr_mg_dl, female):
23      """Cockcroft-Gault estimate in mL/min."""
24      crcl = ((140 - age) * weight_kg) / (72 * scr_mg_dl)
25      return crcl * 0.85 if female else crcl
```

Lines	What it does
9-11	RENAL_RULES says: for each ingredient, below which kidney clearance (mL/min) to alert, how severe and what message. Rules are listed from the lowest threshold up so the most severe applicable one is found first

Lines	What it does
13-20	Metformin (CRITICAL below 30, MAJOR below 45); three NSAIDs (CRITICAL below 30, MODERATE below 60); amoxicillin and clavulanic acid (MAJOR below 30); hydrochlorothiazide (MODERATE below 30). All are placeholders for the pharmacist
22-25	Cockcroft-Gault: ((140 - age) x weight) / (72 x creatinine), multiplied by 0.85 for women. Example: age 68, 58 kg, creatinine 2.1 gives about 28 mL/min

```
27  def parse_kidney_readings(text):
28      """Lines like '2026-10-05, 1.9' -> (sorted [(date, creatinine)], [bad lines])."""
29      from datetime import date
30      readings, bad = [], []
31      for line in text.splitlines():
32          if not line.strip(): continue
33          try:
34              d, v = [x.strip() for x in line.split(",")]
35              value = float(v)
36              if value <= 0: raise ValueError
37              readings.append((date.fromisoformat(d), value))
38          except ValueError:
39              bad.append(line.strip())
40      return sorted(readings), bad
41
42  def latest_reading(readings, on_date):
43      ok = [r for r in readings if r[0] <= on_date]
44      return ok[-1] if ok else None
```

`parse_kidney_readings` turns lines like `2026-10-06, 2.1` into (date, number) pairs, sorts them by date, and collects lines that could not be read so the screen can show an error. `latest_reading` returns the newest reading on or before a given date (or None).

4.2 Parsing durations (lines 46-66)

```
46  # ---------- 2. PARSING WITH DURATION ----------
47  UNIT_DAYS = {"d": 1, "day": 1, "days": 1, "w": 7, "wk": 7, "week": 7, "weeks": 7, "m": 30, "month": 30, "mon ...
48  DUR1 = re.compile(r"\b(?:x|for)\s*(\d+)\s*(days?|d|weeks?|wk|w|months?|m)?\b", re.I)
49  DUR2 = re.compile(r"\b(\d+)\s*(days?|weeks?|months?)\b", re.I)
50
51  def parse_duration(line):
52      """'Tab Combiflame TDS x 5d' -> ('Tab Combiflame TDS', 5). No duration -> None (ongoing)."""
53      for rx in (DUR1, DUR2):
54          m = rx.search(line)
55          if m:
56              unit = (m.group(2) or "d").lower()
57              return (line[:m.start()] + " " + line[m.end():]).strip(), int(m.group(1)) * UNIT_DAYS[unit]
58      return line, None
59
60  def parse_rx_line(line):
61      clean, days = parse_duration(line)
62      p = parse_line(clean)
63      if not p: return None
64      p["raw"] = line.strip()
65      p["days"] = days
66      return p
```

Lines	What it does
47	How many days each unit means: d=1, w=7, m=30
48-49	Two regular expressions (text patterns). The first finds 'x 5d', 'x5 days', 'for 2 weeks'. The second finds '5 days' with no x. A plain number such as the 40 in 'Pan 40' is NOT matched, because a unit or an x/for is required
51-58	parse_duration removes the duration text from the line and returns the cleaned line plus the number of days, or None (ongoing)
60-66	parse_rx_line cleans the duration first, then reuses Phase 1's parse_line for name and frequency, and adds days to the result

Try it (Person B): in `py -3.13` run `from phase2 import parse_rx_line` then `parse_rx_line('Tab Dolo 650 TDS for 2 weeks')`. Predict days first (14). Then try 'x 3d', 'Pan 40 OD', and 'Combiflam TDS'.

4.3 Resolving medicines (lines 68-120)

```
83  def resolve_medicine(name):
84      """Returns a dict with status: verified | generic | proposed | unverified, plus ingredients and confiden ...
85      key = name.lower().strip()
86      if key in ALIAS_TO_BRAND:                                       # exact known spelling
87          b = ALIAS_TO_BRAND[key]
88          return _result("verified", b, BRANDS[b]["ing"], 1.0, "")
89      typed_nums = NUM.findall(name)
90      for w in re.findall(r"[a-z]+", key):                            # generic (chemical) name typed?
91          for ing in known_ingredients():
92              if SequenceMatcher(None, w, ing).ratio() >= 0.88:
93                  if typed_nums:
94                      mg = _num(typed_nums[0])
95                      return _result("generic", f"{ing.title()} {mg:g} mg (generic)", {ing: mg}, 0.9,
96                                     "Generic name recognised; strength taken from the typed text.")
97                  return _result("unverified", name, {}, 0.0, "Generic name recognised but no strength was typ ...
98      b, score, st, sugg = match_brand(name)
99      if st == "unknown":
100         return _result("unverified", name, {}, 0.0, "Not in our database. No safety checks were done for it. ...
101     ings = BRANDS[b]["ing"]
102     brand_nums = NUM.findall(b)
103     if typed_nums and brand_nums and set(typed_nums) != set(brand_nums):     # e.g. 'Dolo 500' vs 'Dolo 650'
104         if len(ings) == 1:
105             only = next(iter(ings)); mg = _num(typed_nums[0])
106             return _result("proposed", f"{b} variant: {only} {mg:g} mg", {only: mg}, score * 0.9,
107                            f"Typed strength ({typed_nums[0]}) differs from {b}. Strength taken from the type ...
108         return _result("proposed", b, ings, score * 0.8,
109                        f"Typed strength ({typed_nums[0]}) differs from {b} in our database: dose uncertain." ...
110     if st == "ok":
```

```
111          return _result("verified", b, ings, score, "")
112      return _result("proposed", b, ings, score, f"Closest brand in our database is {b}; not verified.", sugg)
```

Every typed medicine ends up in exactly one status. The function tries these steps in order:

Step	Test	Status and confidence	Example
1	Exact known spelling (lines 86-88)	verified, 100%	'dolo 650'
2	A known chemical name typed (lines 90-97)	generic, 90% if a strength was typed; unverified if no strength	'Pantoprazole 40'
3	Fuzzy brand match below 0.6 (lines 98-100)	unverified, no composition, a MAJOR alert, overall banner INCOMPLETE	'Zzzz'
4	Strength in text differs from the matched brand (lines 101-109)	proposed; for single-ingredient brands the typed strength is used and confidence is score x 0.9	'Dolo 500' becomes paracetamol 500 mg (before: wrongly 650)
5	Fuzzy score 0.85 or more (lines 110-111)	verified, confidence = score	'Combiflame', 95%
6	Fuzzy score 0.6 to 0.85 (line 112)	proposed with confidence = score	'Pan D' proposed as Pan 40, 75%

> **Be careful when you talk about confidence.** It is a text-similarity score shown as a percentage, **not a statistical probability** that the composition is right. Say 'match confidence' and always label proposals as unverified. A medicine such as Pan D really has a different composition from Pan 40, which is why proposals never produce a green all-clear.

`_result` (lines 74-78) packages the answer and registers a temporary entry in BRANDS for proposed or generic compositions, so the Phase 1 `analyse()` can use them. `make_course` (lines 114-120) adds the doctor, the date and the end date (start + days - 1).

4.4 Checking one period (lines 122-168)

```
126  def _segment_alerts(active, s, patient, readings):
127      out = []
128      items = [{"brand": c["brand_key"], "freq": c["freq"]} for c in active if c["ingredients"]]
129      reading = latest_reading(readings, s)
130      crcl = creatinine_clearance(patient["age"], patient["weight"], reading[1], patient["female"]) if reading ...
131      box = bool(patient.get("kidney_box")) and reading is None
132      totals = {}
133      if items:
134          _, alerts, totals = analyse(items, patient["age"], patient["weight"], patient["risk"], box)
135          for sev, title, detail in alerts:
136              if title.startswith("Duplicate ingredient: "):
137                  ing = title.split(": ", 1)[1]
138                  docs = _doctors_with(active, ing)
139                  if len(docs) > 1:
140                      title = f"Cross-doctor duplicate: {ing}"
141                      detail += f" Prescribed by {' and '.join(docs)}."
142              elif title.startswith("Interaction: "):
143                  a, b = title.split(": ", 1)[1].split(" + ")
144                  da, db = _doctors_with(active, a), _doctors_with(active, b)
145                  if da and db and not set(da) & set(db):
146                      detail += f" {a} from {', '.join(da)}; {b} from {', '.join(db)}: different doctors, so n ...
147              elif title.startswith("Two ") and " belong" in detail:
148                  names = detail.split(" belong")[0].split(" + ")
149                  by_doc = {n: _doctors_with(active, n) for n in names}
150                  if len(set().union(*[set(v) for v in by_doc.values()])) > 1:
151                      detail += " From different doctors: " + "; ".join(f"{n} ({', '.join(v)})" for n, v in by ...
152              out.append((sev, title, detail))
```

Lines	What it does
128, 133-134	Collect the medicines that have a composition and send them to Phase 1's analyse(), which returns duplicates, same-class and interaction alerts, dose limits and daily totals for this period
129-131	Find the latest kidney value for this period and turn it into clearance. The old yes/no kidney checkbox is used only when there are no readings
136-141	If analyse reports 'Duplicate ingredient', check which doctors own it. If more than one: rename to 'Cross-doctor duplicate' and name them
142-146	For interactions: if the two ingredients come from different doctors, add the note that neither may have seen both
147-151	Same idea for 'Two NSAIDs together' (works for two or more ingredients)

```
153      if crcl is not None:                                        # kidney re-check
154          for ing in totals:
155              for below, sev, msg in RENAL_RULES.get(ing, []):
156                  if crcl < below:
157                      out.append((sev, f"Kidney function: {ing}", f"{msg} (CrCl {crcl:.0f} mL/min from creatin ...
158                      break
159      for c in active:                                            # unverified brands
160          if c["status"] == "unverified":
161              out.append(("MAJOR", f"Unverified medicine: {c['name']}",
162                          f"Prescribed by {c['doctor']}. {c['note']} Ask a pharmacist."))
163          elif c["status"] == "proposed":
164              out.append(("MODERATE", f"Unverified composition: {c['name']}",
165                          f"Proposed: {c['label']} ({c['confidence']*100:.0f}% match confidence, not a probabi ...
166          if c["status"] != "unverified" and not c["freq"]:
167              out.append(("MAJOR", f"Frequency missing: {c['name']}", "Daily dose could not be checked."))
168      return out, totals, crcl
```

Lines	What it does
153-158	If there is a kidney value, apply RENAL_RULES to every ingredient taken in this period; stop at the first (most severe) rule that applies
159-167	Unverified medicine: MAJOR alert. Proposed composition: MODERATE alert with the confidence shown. Missing frequency: MAJOR alert, since the dose could not be checked

4.5 Building the ranked timeline (lines 170-210)

```
181  def check_timeline(courses, patient, readings):
182      points = set()
183      for c in courses:
184          points.add(c["start"])
185          if c["end"]: points.add(c["end"] + timedelta(days=1))
186      for d, _ in readings: points.add(d)
187      points = sorted(points)
188      seen, segments = {}, []                # title -> list of (start, end, severity, detail)
189      for i, s in enumerate(points):
190          e = points[i + 1] - timedelta(days=1) if i + 1 < len(points) else None
191          active = [c for c in courses if c["start"] <= s and (c["end"] is None or c["end"] >= s)]
192          if not active: continue
193          alerts, totals, crcl = _segment_alerts(active, s, patient, readings)
194          segments.append({"start": s, "end": e, "active": [c["label"] for c in active], "totals": totals, "cr ...
195          for sev, title, detail in alerts:
196              seen.setdefault(title, []).append((s, e, sev, detail))
197      ranked = []
198      for title, occ in seen.items():
199          top = min(SEV_ORDER[o[2]] for o in occ)
200          peak = [o for o in occ if SEV_ORDER[o[2]] == top]
201          ranked.append({"severity": peak[0][2], "title": title, "detail": peak[0][3],
202                         "ranges": merge_ranges([(o[0], o[1]) for o in occ]),
203                         "peak_ranges": merge_ranges([(o[0], o[1]) for o in peak]),
204                         "escalated": len(peak) < len(occ)})
205      ranked.sort(key=lambda a: (SEV_ORDER[a["severity"]], a["peak_ranges"][0][0], a["title"]))
206      for rank, a in enumerate(ranked, 1): a["rank"] = rank
207      incomplete = any(c["status"] == "unverified" or not c["freq"] for c in courses)
208      assumed = any(c["status"] in ("proposed", "generic") for c in courses)
209      status = "INCOMPLETE" if incomplete else ("ASSUMPTIONS" if assumed else "CHECKED")
210      return {"alerts": ranked, "segments": segments, "courses": courses, "status": status}
```

Lines	What it does
182-187	Change points: every start date, the day after every end date, and every kidney reading date
189-194	For each period: find the active courses, run the period check and remember the period's totals and kidney clearance
195-196	Store every alert under its title, with the dates and severity it appeared at (the same alert shows up in many periods)
197-204	Merge each title into one entry: its highest severity, the detail at that severity, all dates joined together (merge_ranges), the dates at the peak severity, and 'escalated' if it got worse later
205-206	Sort by severity first (Critical to Info), then by the earliest date at that severity, then by title; then number the alerts 1, 2, 3...
207-210	Overall status: INCOMPLETE if any medicine is unverified or has no frequency; ASSUMPTIONS if any composition was proposed or generic; otherwise CHECKED

4.6 The kidney re-check (lines 212-219)

```
212  def recheck_with_new_value(courses, patient, readings):
213      """Re-run the whole history with and without the newest kidney value; return what changed."""
214      readings = sorted(readings)
215      after = check_timeline(courses, patient, readings)
216      before = check_timeline(courses, patient, readings[:-1])
217      old = {(a["severity"], a["title"]) for a in before["alerts"]}
218      new = [a for a in after["alerts"] if (a["severity"], a["title"]) not in old]
219      return after, before, new
```

Runs the whole history twice: once with all kidney readings and once without the newest. Alerts present only in the first run are the ones **caused by the new value**. The screen shows them in a yellow box. This is the 'when a new kidney value arrives, re-check' requirement, in about 8 lines.

5. app2.py in plain words

Part	What it does
Sidebar	Age, weight, sex, liver box, kidney box (only used when no readings), and the kidney readings box. Adding a line there re-runs the check
Three columns	One per doctor: name, prescription date and medicine lines. Defaults hold the demo scenario
Button + session_state	Same trick as Phase 1: remember that Check was clicked so results stay on screen after any change
Banner	Red INCOMPLETE, yellow ASSUMPTIONS, or green only when everything was identified and checked. This closes the Phase 1 gap where unknown medicines still gave a green tick
Timeline grid	A table with a row per medicine and a # on every active day
Medicines table	Doctor, what was typed, how it was identified, status, confidence, ingredients
Alerts	Numbered most urgent first. Critical and Major shown in red or yellow with dates; the rest under 'more notes'
Kidney re-check	Latest value, estimated clearance and the new alerts caused by it
Day-range details	For each period: active medicines, kidney clearance and daily totals; good for mentors who ask 'show me the working'

6. Demo scenario with the exact results

Patient: 68 years, 58 kg, male. Kidney values: 1 Oct creatinine 1.0 and 6 Oct creatinine 2.1. These are the default inputs of app2.py.

Doctor	Date	Medicines typed
Dr A (physician)	1 Oct	Combiflame 1 TDS x 5d; Glycomet 500 BD
Dr B (orthopaedic)	3 Oct	Voveran 50 BD x 5d; Dolo 650 TDS x 5d
Dr C (cardiologist)	4 Oct	Ecosprin 75 OD; Telma 40 OD; Pan D OD x 10d

Rank	Severity	Alert	Dates
1	CRITICAL	Kidney function: diclofenac (clearance 28 mL/min)	3 to 7 Oct; critical from 6 Oct
2	CRITICAL	Kidney function: metformin (not recommended below 30)	6 Oct to ongoing
3	MAJOR	Cross-doctor duplicate: paracetamol (Combiflam + Dolo 650, 2925 mg/day, Dr A and Dr B)	3 to 5 Oct
4	MAJOR	Two NSAIDs together: ibuprofen (Dr A) + diclofenac (Dr B)	3 to 7 Oct
5	MAJOR	Interaction: aspirin + ibuprofen, different doctors (Dr C and Dr A)	4 to 5 Oct
6-11	MODERATE (more notes)	Ibuprofen and paracetamol close to limit; kidney caution for ibuprofen; ARB + NSAID interactions; Unverified composition: Pan D (proposed Pan 40, 75% match confidence)	various

Banner: yellow 'CHECKED WITH ASSUMPTIONS' because Pan D was proposed. Kidney re-check box: 'New alerts caused by the latest kidney value: Kidney function: diclofenac, Kidney function: metformin'.

Suggested 3-minute Phase 2 demo

Time	Show	Say
0:00	The three doctor boxes and the timeline grid	Three doctors, each prescribing correctly in isolation. Nobody sees the whole picture.
0:30	Alert #3	Combiflam from Dr A and Dolo from Dr B overlap on 3 to 5 October: 2925 mg paracetamol a day. We name both doctors.
1:00	Change Dolo's start so it no longer overlaps	Back-to-back courses are not flagged. We only warn when courses really overlap.
1:30	Add a new kidney line in the sidebar (e.g. 2026-10-06, 2.1)	A new kidney report arrives; everything is re-checked and the yellow box shows the new Critical alerts.
2:00	Type 'Tab Zzzz 1 OD' in a box	Unknown medicines are marked unverified, never silently skipped, and the banner turns red.
2:20	Type 'Tab Dolo 500 TDS'	We propose paracetamol 500 mg with a match confidence and label it unverified.
2:40	Point at the ranking	Most urgent first, numbered, with dates; minor notes are collapsed.
2:55	Close	Prototype and decision support; rules need pharmacist review; next steps are more brands and real kidney-based dose tables.

7. Limitations to state yourselves

Limitation	Detail	Possible fix
Medicine with no duration is treated as ongoing	Conservative: may warn longer than needed for a short course	Ask for a duration or default by drug type
Kidney thresholds are placeholders	Only a few drugs and coarse bands; no dose-by-dose adjustment tables	Pharmacist-reviewed table from a standard reference
Cockcroft-Gault only	Uses actual body weight; needs serum creatinine, age, weight, sex; not valid for unstable kidney function	Add eGFR formulas and a warning for old readings
Kidney reading applies from its date onward	Does not warn when the latest reading is very old	Add 'reading older than N days' note
Aspirin is treated as an NSAID	Low-dose aspirin versus ARB gets a MODERATE alert, which a pharmacist may call too cautious	Separate class for low-dose aspirin
Proposed compositions can be wrong	Pan D differs from Pan 40; confidence is a similarity score, not a probability	'Other / not listed' option, more variants, show alternatives
Generic-name detection needs a strength	'Pantoprazole' alone stays unverified	Ask the user for the strength
Same-day timing is ignored	Two medicines on the same day might be taken hours apart	Needs time-of-day data
No real browser test by me	The logic passes 22 checks, but the Streamlit screen is untested in a real browser	Run it and fix any layout or widget errors
Still 17 brands	Coverage is small; unknown medicines are common	Grow data with the pharmacist

8. New questions mentors may ask about Phase 2

#	Question a mentor may ask	Strong, honest answer
1	What is new in Phase 2 compared with Phase 1?	Phase 1 checks one prescription. Phase 2 merges prescriptions from several doctors into a timeline, finds overlaps and cross-doctor duplicates, re-checks when a kidney value arrives, handles unverified brands and ranks alerts.
2	Why does a timeline matter?	Duplicates and interactions only matter when medicines are taken at the same time. The timeline checks exactly the overlapping days.
3	How do you build the timeline?	Each medicine is a course with start and end dates. We cut the history at every start, end and kidney-reading date. Within each period the active set is fixed, so we run the same analysis on it.
4	What is a cross-doctor duplicate?	The same ingredient prescribed by two different doctors during overlapping dates, for example Combiflam from one doctor and Dolo 650 from another.
5	What if two courses do not overlap?	No duplicate is flagged. This is covered by an automatic test.
6	What if no duration is written?	The medicine is treated as ongoing, which is the cautious choice. We say so and could ask for a duration.
7	How do you re-check when a new kidney value arrives?	We run the whole history with and without the newest reading and show the alerts that appear only because of it.

#	Question a mentor may ask	Strong, honest answer
8	How do you estimate kidney function?	With the Cockcroft-Gault formula from age, weight, sex and serum creatinine, giving creatinine clearance in mL/min.
9	Is Cockcroft-Gault perfect?	No. It is an estimate, not valid in unstable kidney function, and the thresholds in our table are placeholders to be confirmed by a pharmacist.
10	Which drugs have kidney rules?	Metformin, three NSAIDs (ibuprofen, diclofenac, aceclofenac), amoxicillin, clavulanic acid and hydrochlorothiazide. It is a small illustrative set.
11	What happens with an unknown brand?	It is marked unverified, gets a MAJOR alert, is not analysed, and the banner turns red as INCOMPLETE.
12	What does 'propose a likely composition with confidence' mean in your tool?	When a name is close but not exact, we propose the closest known composition and show a match confidence from the text similarity. It is labelled unverified.
13	Is that confidence a probability?	No. It is a similarity score shown as a percentage. We do not claim statistical calibration.
14	What did you do about Dolo 500 being matched to Dolo 650?	If the typed number differs from the brand's strength, we propose a variant using the typed strength (paracetamol 500 mg), lower the confidence and mark it unverified.
15	What if the user types only a chemical name?	If it includes a strength we accept it as a generic composition with 90 percent match confidence. Without a strength it stays unverified.
16	How do you rank alerts?	By severity from Critical to Info, then by the earliest date at that severity, then by title. The result is numbered.
17	How do you avoid repeating the same alert for every day?	We merge alerts with the same title across all periods into one entry with its date ranges and its peak severity.
18	How is alert fatigue handled in Phase 2?	Merged alerts, ranking, collapsed minor notes and an overall status banner. We would measure false alarms on a larger reviewed set.
19	Why did you not change engine.py?	We reused it as it is, so Phase 1 stays tested and stable. Phase 2 wraps it by calling analyse for each period.
20	How did you test Phase 2?	22 automatic checks in evaluate_phase2.py: overlap and non-overlap, same brand from two doctors, kidney re-check, unverified brands, the Dolo 500 case, generic names, ranking, durations and a safe history.
21	Does it handle more than three doctors?	Yes. Each doctor is just a label on a course; the screen shows three boxes but the logic accepts any number.
22	What is the biggest weakness?	Small data and placeholder kidney thresholds. The method is general but the knowledge base needs expert growth and review.
23	Could this be used in a real hospital?	Not as it is. It would need clinical validation, a much larger reviewed database, privacy compliance and regulatory review.
24	What would you do next?	More brands and interactions with pharmacist review, drug-by-drug kidney tables, warnings for old kidney values, and an 'other / not listed' option for unknown brands.

9. Who does what now

Person	Tasks	Time
A (data)	Ask the pharmacist to review RENAL_RULES (lines 13-20), the four older rules and the demo scenario results. Add 5 more brands with kidney relevance (e.g. Telma-H, Augmentin 625 already present)	45 min
B (code)	Run app2.py and evaluate_phase2.py on your laptop; read section 4 of this guide; fix any error; make one improvement (see Try it exercises)	60 min
C (tests and pitch)	Add 5 new checks to evaluate_phase2.py (copy a block, change the data); rebuild the Phase 2 slides and rehearse the 3-minute demo	60 min

Update your pitch

- Add one slide: 'Phase 2: the whole patient history', with the timeline grid screenshot and alert #3 (cross-doctor duplicate).
- Keep Phase 1 slides: they are the engine behind Phase 2.
- In the results slide, add: 22/22 Phase 2 checks passed, and say what they cover.
- In limits, state the placeholder kidney thresholds and that you have not tested with real clinical data.

Starter exercises

- **Easy:** add a kidney rule for another drug in RENAL_RULES (after the pharmacist agrees) and add a check in evaluate_phase2.py for it.
- **Medium:** make a medicine with no duration default to 30 days and an INFO note, and see which tests change.
- **Medium:** add 'reading older than 30 days' as an INFO alert in _segment_alerts.
- **Stretch:** add an 'Other / not listed' choice so a user can mark a medicine as unverified even when a proposal exists.

> **Reminder:** compositions, dose limits, interaction severities and kidney thresholds are illustrative until your pharmacist confirms them. Do not present results as medical advice, and do not say 'safe': say 'no critical or major alerts found in our database'.